

CAPÍTULO 1

ESTRUCTURA DE DATOS

FUNDAMENTOS

Clases, referencias y complejidad

Una red social paso a paso en Python

EDICIÓN PARA ESTUDIANTES

Conceptos, ejemplos y práctica guiada



DATOS

Organizar y relacionar



ALGORITMOS

Comprender el costo



PYTHON

Construir y practicar



Estructura de Datos

Capítulo 1 - Fundamentos

Manual de estudio y práctica con Python

Basado en la presentación «Fundamentos de Estructuras de Datos», de Ing. Leonardo Tavarez, y en la transcripción de la clase práctica sobre una red social.

El contenido se presenta como una lección para estudiar de forma independiente: las explicaciones se reorganizan, los ejemplos se completan y las actividades permiten comprobar lo aprendido.

OBJETIVO DEL CAPÍTULO

Relacionar las decisiones de almacenamiento con el costo de los algoritmos y construir una simulación que utilice objetos, listas, diccionarios y referencias.

Conocimientos previos. Variables, condicionales, ciclos y funciones básicas. Se repasan las clases antes de comenzar el proyecto.

Herramientas. Python 3.10 o posterior, un editor y una terminal. Sin dependencias externas.

Edición de estudio - Septiembre de 2026

Contenido

Lee las secciones conceptuales antes de programar. Crea cada archivo cuando se indique, ejecuta el proyecto y resuelve las actividades antes de consultar las respuestas. El índice incluye enlaces internos.

1.1 Pensar en estructuras de datos	4
1.2 Clasificación y operaciones fundamentales	5
1.3 Medir el crecimiento del costo	7
1.4 Big O, Omega y Theta	8
1.5 Órdenes de crecimiento habituales	9
1.6 Clases, objetos y referencias en Python	11
1.7 Diseñar la pequeña red social	12
1.8 Preparar los archivos y las anotaciones	14
1.9 Implementar Comment y Post	16
1.10 Implementar User y la amistad bidireccional	18
1.11 Registrar y consultar usuarios	19
1.12 Construir el timeline paso a paso	20
1.13 Ejecutar una simulación completa	22
1.14 Analizar el costo del proyecto	23
1.15 Comprobar el comportamiento y resolver errores	25
1.16 Práctica integradora	27
1.17 Respuestas y solución orientativa	28
1.18 Síntesis, glosario y fuentes	30

Fundamentos de estructuras de datos

1

Ruta de aprendizaje

Organización de datos → complejidad → clases y referencias → red social en Python → análisis y práctica.

Una red social reúne personas, publicaciones y relaciones. Ese escenario nos permitirá estudiar tanto las estructuras que conservan información como los algoritmos que la transforman en un resultado útil.

1.1 Pensar en estructuras de datos

Un programa no solo necesita instrucciones: también necesita decidir cómo conservar y relacionar la información. Una **estructura de datos** es una organización de datos que permite realizar operaciones como insertar, buscar, eliminar y recorrer. Elegirla cambia el trabajo que debe hacer un algoritmo para resolver el mismo problema.

Imagina una red social con miles de personas. Si guardas sus usuarios en una lista, encontrar a «ana» puede exigir revisar muchos elementos. Si los guardas en un diccionario cuya clave es el nombre de usuario, puedes consultar esa clave directamente. El resultado funcional es el mismo; el costo de obtenerlo cambia.

La eficiencia tiene dos dimensiones que estudiaremos durante el curso: tiempo de ejecución y uso de memoria. Una estructura también influye en la claridad del código, en la facilidad de mantenimiento y en la capacidad de la aplicación para crecer. Sin embargo, una estructura no garantiza por sí sola que un sistema sea correcto: todavía necesitamos reglas y algoritmos adecuados.

1.1.1 Del problema a la elección

Antes de programar, identifica qué información existe, cómo se relaciona y cuáles serán las operaciones frecuentes. No hay una estructura que sea la mejor para todas las tareas.

Pregunta de diseño	Consecuencia práctica
¿Se buscará por una clave única?	Considerar un diccionario para localizar el dato por esa clave.
¿Importa la posición o la secuencia?	Considerar una lista para conservar y recorrer elementos.
¿Se atiende en orden de llegada?	Pensar en una cola: primero en entrar, primero en salir.
¿Se necesita deshacer lo último?	Pensar en una pila: último en entrar, primero en salir.
¿Hay jerarquías o conexiones?	Considerar árboles o grafos para representar relaciones.

En la práctica guiada, los usuarios y los amigos se buscarán por username. Las publicaciones y los comentarios se recorrerán en secuencia. Por eso combinaremos diccionarios y listas dentro de un mismo proyecto.

Actividad 1. En un sistema de biblioteca, ¿guardarías los libros por posición o por código único para consultar rápidamente un ejemplar? Justifica tu elección e identifica otra operación que podría necesitar una estructura distinta.

1.2 Clasificación y operaciones fundamentales

1.2.1 Estructuras lineales y no lineales

En una estructura **lineal**, los elementos forman una secuencia lógica. Arreglos, listas, pilas y colas pertenecen a esta familia. Los elementos de los extremos no tienen necesariamente un vecino a ambos lados. Además, la organización lógica no implica que todas las implementaciones ocupen posiciones contiguas en memoria.

Una estructura **no lineal** representa ramificaciones o conexiones más generales. Un árbol organiza una jerarquía, como carpetas dentro de carpetas. Un grafo organiza vértices y aristas, como personas y relaciones de amistad. Un árbol es una clase particular de grafo; en este capítulo los distinguimos por su uso didáctico.

SECUENCIA



Lista, pila o cola: reglas de acceso distintas.

JERARQUIA

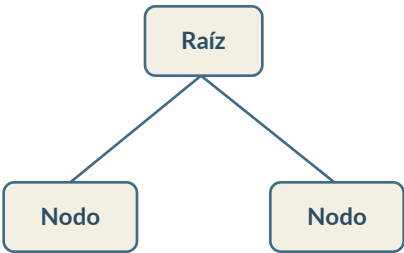


Figura 1.1 Organización secuencial y organización jerárquica.

Una pila y una cola describen, principalmente, reglas de acceso. Pueden construirse utilizando otras estructuras. Una lista de Python, por su parte, se implementa como un arreglo dinámico de referencias; no debe confundirse con una lista enlazada.

1.2.2 Cuatro operaciones básicas

Operación	Qué hace	Ejemplo en la red social
Insertar	Agrega un elemento.	Registrar un usuario o añadir un comentario.
Eliminar	Quita un elemento o una relación.	Retirar una amistad.
Buscar	Localiza un elemento según un criterio.	Consultar un username.
Recorrer	Visita los elementos para procesarlos.	Mostrar todas las publicaciones seleccionadas.

El costo depende de la operación y de la estructura. Acceder a una posición conocida de una lista es distinto de buscar un valor dentro de ella. Del mismo modo, quitar un elemento del final no tiene el mismo costo que eliminarlo al principio y desplazar los restantes.

1.2.3 Estructuras de datos y patrones de diseño

Una estructura de datos organiza información; un patrón de diseño describe una solución recurrente de organización o colaboración del software. Una lista responde a cómo conservar una secuencia. Observer responde a cómo notificar a objetos interesados cuando ocurre un cambio. Factory y Singleton son otros ejemplos de patrones mencionados en la presentación.

Ambos ayudan a tomar decisiones de diseño y pueden combinarse. Por ejemplo, un mecanismo de notificaciones puede conservar sus observadores en una lista. Elegir una lista no equivale a implementar Observer, ni crear una clase de servicio convierte automáticamente el proyecto en un patrón específico.

Actividad 2. Clasifica estas necesidades: historial para deshacer, fila de impresión, carpetas de un equipo y amistades de una red social. Indica pila, cola, árbol o grafo y explica la regla que te llevó a elegir.

1.3 Medir el crecimiento del costo

1.3.1 Tamaño de entrada y modelo de análisis

La **complejidad algorítmica** estudia cómo crece el costo de un algoritmo al aumentar su entrada. Primero define qué significa n . Puede ser el número de usuarios, de publicaciones o de elementos de una lista. Si intervienen colecciones distintas, usa variables distintas.

Contar operaciones permite comparar el crecimiento sin depender de un equipo concreto. No predice segundos exactos: el hardware, el intérprete, los datos y las constantes también afectan al tiempo observado. Medir tiempos reales complementa el análisis; no lo sustituye.

Si buscar un nombre exige recorrer una lista de n usuarios, en el peor caso revisaremos n elementos. Con el doble de usuarios podemos necesitar aproximadamente el doble de comparaciones. Esa relación es lineal.

1.3.2 Complejidad temporal y espacial

La **complejidad temporal** describe el crecimiento del trabajo. La **complejidad espacial** describe el crecimiento de la memoria necesaria. Conviene indicar si hablamos de memoria total o de **memoria auxiliar**, que excluye la entrada. En los análisis de este capítulo especificaremos memoria auxiliar y, cuando corresponda, memoria de salida.

Listado 1.1

```
valores = [4, 7, 2, 9]
total = 0
for valor in valores:
    total += valor
print(total) # 22
```

La suma visita cada elemento una vez: tiempo $\Theta(n)$. El acumulador utiliza espacio auxiliar $O(1)$, bajo el modelo simplificado de números de tamaño acotado. No contamos la lista de entrada como memoria adicional del algoritmo.

Listado 1.2

```
dobles = []
for valor in valores:
    dobles.append(valor * 2)
```

Este segundo recorrido también emplea tiempo $\Theta(n)$, pero construye una lista de n resultados: espacio adicional $\Theta(n)$. Dos algoritmos pueden tener el mismo crecimiento temporal y necesidades de memoria diferentes.

Actividad 3. Un algoritmo recorre una lista y conserva únicamente su valor máximo. Otro crea una copia completa antes de buscarlo. Compara su tiempo y su memoria auxiliar. Supón que ambos realizan un solo recorrido para encontrar el máximo.

1.4 Big O, Omega y Theta

Estas notaciones expresan **cotas de crecimiento** cuando la entrada es suficientemente grande. No son unidades de tiempo ni nombres automáticos del peor, mejor y promedio de los casos.

Notación	Interpretación	Ejemplo para $f(n) = 3n + 2$
$O(g(n))$	Cota superior asintótica.	$f(n)$ es $O(n)$.
$\Omega(g(n)), \Omega$	Cota inferior asintótica.	$f(n)$ es $\Omega(n)$.
$\Theta(g(n)), \Theta$	Cota ajustada: superior e inferior del mismo orden.	$f(n)$ es $\Theta(n)$.

Decir que $f(n)$ es $O(g(n))$ significa que, desde algún tamaño n_0 , $f(n)$ no supera una constante positiva multiplicada por $g(n)$. Para Omega ocurre la desigualdad inversa. Para Theta existen ambas cotas. Esta formulación explica por qué se omiten factores constantes y términos de menor crecimiento [R1].

Por ejemplo, $3n + 2$ queda acotado entre $3n$ y $5n$ para n mayor o igual que 1. Su orden ajustado es $\Theta(n)$. También es $O(n^2)$, pero esa cota superior aporta menos precisión.

1.4.1 Casos de entrada y cotas son decisiones distintas

En una búsqueda secuencial que termina al encontrar el objetivo, el mejor caso visita el primer elemento y cuesta $\Theta(1)$. El peor caso visita todos y cuesta $\Theta(n)$. El caso promedio requiere declarar una distribución: si el objetivo está presente y es igualmente probable en cada posición, el número medio de comparaciones es $(n + 1) / 2$, también $\Theta(n)$.

Precisión conceptual. Big O no significa necesariamente «peor caso»; Omega no significa «mejor caso» y Theta no significa «caso promedio». Podemos aplicar cualquiera de estas cotas a una función de costo del mejor caso, del peor caso o del promedio. Primero elegimos el caso que analizamos; después expresamos su crecimiento.

1.4.2 Reglas prácticas para comenzar

- Identifica los tamaños de entrada y la operación que cuentas.
- Suma el costo de pasos consecutivos; conserva los términos dominantes.
- Multiplica iteraciones cuando realmente se ejecuta el producto de dos recorridos completos.
- Incluye el trabajo oculto en operaciones como copiar, ordenar o recorrer una colección.
- Distingue costo promedio, peor caso y costo amortizado cuando corresponda.

El costo **amortizado** distribuye el trabajo de una secuencia de operaciones. Por ejemplo, una inserción al final de una lista puede requerir redimensionar su almacenamiento, pero el costo amortizado de `append` es $O(1)$. Esto no equivale a afirmar que cada llamada individual cuesta siempre lo mismo [R2].

1.5 Órdenes de crecimiento habituales

Orden	Idea	Ejemplo representativo
$O(1)$	El trabajo no crece con n .	Acceder a una posición válida de una lista.
$O(\log n)$	El problema se reduce por un factor en cada paso.	Búsqueda binaria en una secuencia ordenada.
$O(n)$	Se visita una cantidad proporcional a la entrada.	Recorrer una lista completa.
$O(n \log n)$	Crecimiento habitual de ordenamientos eficientes.	Ordenar comparando elementos.
$O(n^2)$	Se procesa un número cuadrático de combinaciones.	Comparar cada elemento con todos los demás.
$O(2^n)$	El número de posibilidades se duplica por elemento.	Explorar todas las selecciones de n decisiones binarias.

La búsqueda binaria requiere que los datos estén ordenados y que podamos consultar con eficiencia la posición intermedia. No basta con aplicar esa idea a cualquier colección desordenada.

Para visualizar el crecimiento, evaluemos funciones representativas. Los números siguientes son valores matemáticos, no tiempos medidos ni cantidades universales de instrucciones.

Función	n = 10	n = 100
1	1	1
$\log_2(n)$	Aproximadamente 3,32	Aproximadamente 6,64
n	10	100
$n \log_2(n)$	Aproximadamente 33,22	Aproximadamente 664,39
n^2	100	10 000
2^n	1 024	Aproximadamente $1,27 \times 10^{30}$

1.5.1 Dos bucles no bastan para afirmar $O(n^2)$

Listado 1.3

```
for i in range(n):
    for j in range(n):
        procesar(i, j)
```

Si procesar tiene costo constante, se realizan n^2 llamadas. En cambio, recorrer publicaciones y después los comentarios de cada una depende de dos cantidades: el número de publicaciones y la suma de sus comentarios. No se deben inventar n comentarios por publicación si el problema no lo establece.

Actividad 4. Determina el número de visitas para: a) dos recorridos consecutivos de n elementos; b) un recorrido de n elementos con tres pasos internos constantes; c) todos los pares de una colección de n elementos. Expresa el orden ajustado.

1.6 Clases, objetos y referencias en Python

Una **clase** define atributos y comportamiento. Un **objeto** es una instancia concreta. En nuestra red social, User describe a un usuario; leo y ana serán objetos distintos creados a partir de esa clase.

Listado 1.4

```
class Perfil:
    def __init__(self, nombre: str) -> None:
        self.nombre = nombre

    def saludar(self) -> str:
        return f"Hola, soy {self.nombre}"

perfil = Perfil("Ana")
print(perfil.saludar()) # Hola, soy Ana
```

`__init__` inicializa la instancia. `self` es el parámetro que recibe esa instancia en un método normal; por convención lo llamamos así. `self.nombre` pertenece al objeto. La anotación `-> str` indica el retorno esperado de `saludar`. Una función o método que termina sin un `return` explícito devuelve `None`.

`None` es un objeto especial que representa ausencia de valor. `-> None` comunica que no se espera un resultado útil. No convierte al método en una categoría separada: un método de Python puede devolver datos o devolver `None`.

1.6.1 Referenciar no es copiar

En Python, los nombres se vinculan a objetos. Una asignación como `otro = perfil` crea otra referencia al mismo objeto; no duplica automáticamente sus atributos. Esta idea permite que un usuario sea accesible desde el registro de la red, desde el diccionario de amigos y desde la autoría de una publicación [R3].

Listado 1.5

```
otro = perfil
print(otro is perfil) # True
otro.nombre = "Ana Maria"
print(perfil.nombre) # Ana Maria

otro = Perfil("Luis")
print(perfil.nombre) # Ana Maria
```

La primera modificación cambia el objeto compartido. La última asignación cambia a qué objeto apunta el nombre `otro`; no modifica el vínculo de `perfil`. El operador `is` compara identidad; `==` compara igualdad según el comportamiento definido por el tipo.

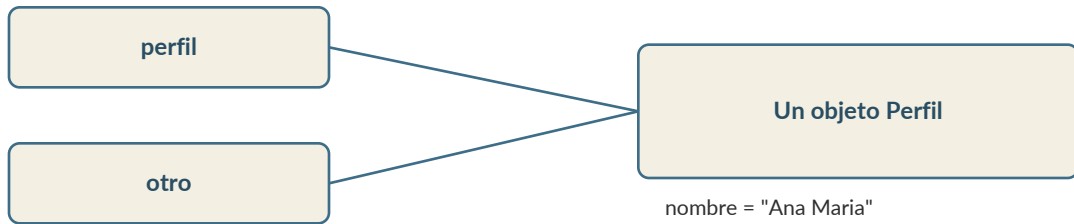


Figura 1.2 Dos nombres permiten acceder al mismo objeto.

Precisión conceptual. En este manual hablaremos de referencias. La explicación del video usa «punteros» como aproximación, pero en Python no manipulamos direcciones de memoria ni hacemos aritmética de punteros como en C.

Una copia superficial de una lista crea un contenedor nuevo que sigue conteniendo referencias a los mismos elementos. Esto será útil en el timeline: reunir publicaciones no exige duplicar los objetos Post.

Actividad 5. Si dos variables contienen el mismo User y una modifica su name, ¿qué nombre observará la otra? ¿Qué sucede si en lugar de modificar name asignas un User nuevo a una de las variables?

1.7 Diseñar la pequeña red social

Construiremos una simulación en memoria llamada **Social ITLA**. Tendrá usuarios, amistades bidireccionales, publicaciones, comentarios y un timeline. El timeline de una persona reunirá sus publicaciones y las de sus amigos directos, ordenadas de la más reciente a la más antigua.

El proyecto no incluye inicio de sesión, base de datos ni interfaz web. Al terminar el proceso, los datos se pierden. Esta decisión permite observar la organización de objetos sin añadir infraestructura ajena al capítulo.

Componente	Datos principales	Responsabilidad
Comment	author, content, created_at	Representar un comentario.
Post	author, content, created_at, comments	Representar una publicación y añadir comentarios.
User	username, name, posts, friends	Publicar y establecer amistades.
SocialNetwork	name, users	Registrar, buscar y coordinar el timeline.

1.7.1 Reglas que mantendremos

- Cada username identifica a un solo usuario dentro de la red.
- Un usuario no puede agregarse a sí mismo como amigo.
- Si A es amigo de B, B también es amigo de A.
- Cada publicación conserva una referencia a su autor.
- Cada comentario pertenece a la lista de comentarios de una publicación.
- El límite del timeline es un entero no negativo y se aplica a publicaciones, no a comentarios.

En este ejemplo, username se trata como un identificador estable. Cambiarlo directamente después del registro dejaría claves antiguas en los diccionarios. name, en cambio, puede cambiar sin alterar esas claves. Centralizar una futura operación de cambio de username sería una ampliación del diseño.

1.7.2 Por qué listas y diccionarios

Usaremos `dict[str, User]` para buscar usuarios y amigos por username. Una consulta por clave suele costar $O(1)$ en promedio; el peor caso puede ser $O(n)$. Este análisis supone claves con costos de hash y comparación acotados. Un diccionario no garantiza que todos los accesos sean constantes en cualquier circunstancia [R2].

Usaremos `list[Post]` y `list[Comment]` porque añadiremos elementos al final y los recorreremos. Un diccionario conserva orden de inserción, pero no ordena por fecha. Por eso el timeline necesitará una ordenación explícita.

Las amistades forman un **grafo no dirigido**: cada User es un vértice y cada amistad es una arista. Los diccionarios friends actúan como colecciones de vecinos. Esta representación conecta la teoría de la presentación con el proyecto del video.

1.8 Preparar los archivos y las anotaciones

Utiliza Python 3.10 o posterior y un editor de texto. El proyecto solo necesita la biblioteca estándar. Crearemos un paquete llamado `social`; puedes colocar su carpeta raíz dentro de `src` si mantienes ese directorio como punto de ejecución.

Para evitar ambigüedades, en las instrucciones siguientes la carpeta de trabajo se llama `proyecto_social`. Dentro de ella se encuentra `social`. Los archivos `__init__.py` pueden estar vacíos.

Ruta relativa a <code>proyecto_social</code>	Contenido
<code>social/__init__.py</code>	Marca el paquete principal.
<code>social/models/__init__.py</code>	Marca el subpaquete de modelos.
<code>social/models/comment.py</code>	Clase <code>Comment</code> .
<code>social/models/post.py</code>	Clase <code>Post</code> .
<code>social/models/user.py</code>	Clase <code>User</code> .
<code>social/services/__init__.py</code>	Marca el subpaquete de servicios.
<code>social/services/social_network.py</code>	Clase <code>SocialNetwork</code> .
<code>social/main.py</code>	Demostración ejecutable.

Los nombres de archivo y función siguen `snake_case`: palabras en minúsculas separadas por guion bajo. Las clases usan nombres como `SocialNetwork`. En una importación relativa, `.comment` se refiere al módulo hermano dentro del paquete actual y `..models` al subpaquete `models` desde el paquete padre.

1.8.1 Anotaciones y dependencias entre módulos

Las anotaciones describen tipos esperados para lectores, editores y analizadores. Por sí solas no validan ni convierten los valores al ejecutar Python [R4]. En este proyecto utilizaremos `list` y `dict` con parámetros de tipo, en lugar de las formas antiguas `List` y `Dict` de `typing`.

`User` necesita crear `Post`, y `Post` necesita describir a `User` como tipo de su autor. Si ambos módulos se importaran mutuamente al cargarse, podríamos provocar una importación circular. Separaremos las dependencias necesarias al ejecutar de las necesarias solo para analizar tipos.

Listado 1.6

```
from __future__ import annotations
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .user import User
```

La primera línea pospone la evaluación de anotaciones. `TYPE_CHECKING` es falso durante la ejecución normal y se trata como verdadero durante el análisis estático. Así podemos describir `User` sin importarlo en ese momento. Esto sirve cuando el nombre se usa únicamente como anotación; no funcionaría para ejecutar `User(...)` dentro de ese módulo sin una importación real [R4].

Precisión conceptual. Esta técnica evita el ciclo de importaciones concreto de los modelos; no elimina todas las dependencias circulares ni todos los errores de tipos. Los ciclos entre objetos, como dos amigos que se referencian, son otro concepto distinto.

1.9 Implementar Comment y Post

1.9.1 Archivo social/models/comment.py

Listado 1.7

```
from __future__ import annotations

from dataclasses import dataclass
from datetime import datetime
from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from .user import User

@dataclass
class Comment:
    author: User
    content: str
    created_at: datetime
```

Comment concentra los datos de un comentario. @dataclass genera métodos habituales como `__init__`, `__repr__` y `__eq__`, según sus opciones. Aquí el constructor generado recibe `author`, `content` y `created_at`, en ese orden.

Una dataclass **sí puede tener métodos propios**. Elegimos esta forma porque el modelo es pequeño, no porque exista una prohibición de añadir comportamiento. Las comparaciones de orden no se generan por defecto: la opción `order` es `False` [R5].

1.9.2 Archivo social/models/post.py

Listado 1.8

```
from __future__ import annotations

from datetime import datetime, timezone
from typing import TYPE_CHECKING
from .comment import Comment

if TYPE_CHECKING:
    from .user import User

class Post:
    def __init__(self, author: User, content: str) -> None:
        self.author = author
        self.content = content
        self.created_at = datetime.now(timezone.utc)
        self.comments: list[Comment] = []

    def add_comment(self, author: User, content: str) -> None:
        comment = Comment(
            author=author,
            content=content,
            created_at=datetime.now(timezone.utc),
        )
        self.comments.append(comment)

    def __repr__(self) -> str:
        return (
            f"Post(author={self.author.username!r}, "
            f"content={self.content!r}, "
            f"comments={len(self.comments)})"
        )
```

Cada Post conserva una lista nueva de comentarios. Crear `self.comments` dentro de `__init__` evita compartir accidentalmente la misma lista entre todas las publicaciones. Lo mismo haremos con los amigos y las publicaciones de cada usuario.

`add_comment` recibe un autor y un texto, construye el `Comment` y lo incorpora con `append`. La fecha se calcula al crear el comentario. Usamos fechas con zona UTC para que las comparaciones temporales de este proyecto sean consistentes.

`__repr__` devuelve una representación útil para inspección y depuración. `!r` aplica `repr` al valor, de modo que las cadenas se muestran con comillas. El método muestra cantidades y datos breves, en lugar de recorrer amigos o autores recursivamente. Cuando no se define `__str__`, `print` puede utilizar `__repr__` [R3].

Actividad 6. Explica por qué `Comment` se importa normalmente en `post.py`, mientras que `User` se importa dentro de `if TYPE_CHECKING`. Señala la línea que necesita crear un objeto durante la ejecución.

1.10 Implementar User y la amistad bidireccional

1.10.1 Archivo social/models/user.py

Listado 1.9

```
from __future__ import annotations

from .post import Post

class User:
    def __init__(self, username: str, name: str) -> None:
        self.username = username
        self.name = name
        self.posts: list[Post] = []
        self.friends: dict[str, User] = {}

    def add_friend(self, other: User) -> None:
        if other.username == self.username:
            raise ValueError("No puedes agregarte a ti mismo.")
        self.friends[other.username] = other
        other.friends[self.username] = self

    def create_post(self, content: str) -> Post:
        post = Post(author=self, content=content)
        self.posts.append(post)
        return post

    def __repr__(self) -> str:
        return (
            f"User(username={self.username!r}, "
            f"name={self.name!r}, "
            f"friends={len(self.friends)}, "
            f"posts={len(self.posts)})"
        )
```

La primera asignación de `add_friend` guarda a `other` dentro de los amigos del usuario actual. La segunda guarda al usuario actual dentro de los amigos de `other`. Ambas almacenan referencias a objetos existentes. No se crean usuarios nuevos.

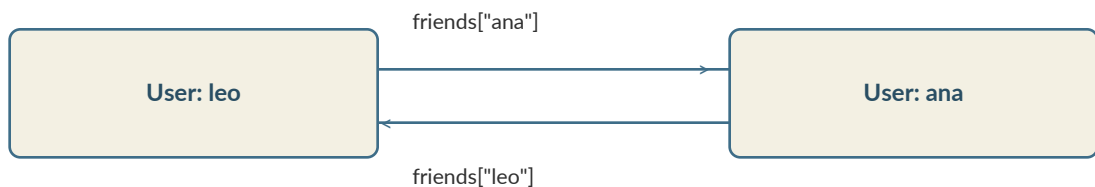


Figura 1.3 Una amistad se conserva en los dos diccionarios.

Si repetimos la misma amistad, las claves existentes se actualizan con las mismas referencias y no aparecen duplicados. Esta propiedad depende de que trabajemos con los usuarios únicos registrados por el servicio.

`create_post` utiliza `self` como autor, conserva el `Post` en la lista personal y lo devuelve. Ese retorno permite guardar una variable como `p1` y agregarle comentarios después. Como `p1` y el elemento de `posts` referencian el mismo objeto, los cambios serán visibles desde ambos accesos.

Actividad 7. Después de `leo.add_friend(ana)`, predice el resultado de `leo.friends['ana']` is `ana` y `ana.friends['leo']` is `leo`. ¿Qué regla se rompería si eliminaras la segunda asignación de `add_friend`?

1.11 Registrar y consultar usuarios

La clase `SocialNetwork` coordina operaciones que abarcan varios usuarios. El modelo `User` sabe establecer una amistad; el servicio sabe localizar a las dos personas a partir de sus nombres de usuario. Separar estas responsabilidades evita repetir búsquedas en cada operación.

1.11.1 Archivo `social/services/social_network.py`: primera parte

Escribe este bloque y continúa la misma clase con el método de la sección siguiente. No crees una segunda clase `SocialNetwork`.

Listado 1.10

```
from __future__ import annotations

from ..models.post import Post
from ..models.user import User

class SocialNetwork:
    def __init__(self, name: str) -> None:
        self.name = name
        self.users: dict[str, User] = {}

    def register_user(self, username: str, name: str) -> User:
        if username in self.users:
            raise ValueError("El username ya esta registrado.")
        user = User(username, name)
        self.users[username] = user
        return user

    def get_user(self, username: str) -> User:
        if username not in self.users:
            raise ValueError(f"Usuario no registrado: {username}")
        return self.users[username]

    def add_friends(self, first: str, second: str) -> None:
```

Listado 1.10 - continuación

```
user1 = self.get_user(first)
user2 = self.get_user(second)
user1.add_friend(user2)
```

username in self.users consulta las **claves** del diccionario. Antes de crear el usuario, verificamos que su identificador no exista. Así evitamos reemplazar accidentalmente un objeto que ya tiene amistades o publicaciones.

get_user tiene un contrato sencillo: devuelve un User o lanza ValueError. No devuelve None. Por eso add_friends no necesita volver a comprobar si recibió None después de llamar a get_user. Si falla una búsqueda, la excepción interrumpe el método antes de modificar la amistad.

El ejemplo asume nombres no vacíos y datos bien formados para concentrarse en estructuras. Puedes añadir validaciones de contenido como ampliación. Las comprobaciones explícitas de este código cubren duplicados, inexistencia de usuarios y autoamistad.

1.12 Construir el timeline paso a paso

El timeline es una lista de líneas de texto. Para construirla debemos buscar al usuario, reunir publicaciones, ordenar, limitar y finalmente representar las publicaciones elegidas junto con sus comentarios.

1.12.1 Archivo social/services/social_network.py: continuación

Añade este método dentro de SocialNetwork, con la misma sangría que register_user. El nombre Post ya fue importado al principio del archivo.

Listado 1.11

```
def timeline(self, username: str, limit: int = 10) -> list[str]:
    if limit < 0:
        raise ValueError("El limite no puede ser negativo.")
    user = self.get_user(username)
    feed: list[Post] = []
    feed.extend(user.posts)
    for friend in user.friends.values():
        feed.extend(friend.posts)

    feed.sort(key=lambda post: post.created_at, reverse=True)
    selected = feed[:limit]
    lines = [f"Timeline de @{username} en {self.name}"]
    for post in selected:
        date = post.created_at.strftime("%Y-%m-%d %H:%M")
        lines.append(
            f"{date} UTC | @{post.author.username}: "
            f"{post.content}"
        )
```

Listado 1.11 - continuación

```
    )
    for comment in post.comments:
        date = comment.created_at.strftime("%Y-%m-%d %H:%M")
        lines.append(
            f" {date} UTC | @{comment.author.username}: "
            f"{comment.content}"
        )
    return lines
```

1.12.2 Reunir, ordenar y recortar

extend añade cada elemento de una colección. Si usáramos `append(user.posts)`, agregaríamos una lista completa como un único elemento y `feed` dejaría de ser una lista plana de `Post`. `friends.values()` recorre objetos `User`; recorrer directamente `friends` daría sus claves.

La lista `feed` contiene referencias a publicaciones existentes. Ordenarla cambia el orden de esta lista temporal; no cambia el orden de las listas `posts` de los autores.

La expresión `lambda post: post.created_at` produce la clave de ordenación. `reverse=True` solicita orden descendente. Si dos publicaciones tienen la misma fecha, la ordenación estable conserva su orden relativo previo [R6].

`feed[:limit]` crea una lista con las primeras publicaciones hasta el límite. Con `limit` igual a 0 no selecciona ninguna; el método devuelve únicamente el encabezado. Si hay menos publicaciones que el límite, devuelve las disponibles. Un límite negativo se rechaza para evitar que el corte tenga otro significado.

1.12.3 Representar el resultado

Cada publicación produce una línea y cada comentario otra línea con sangría. Los comentarios se conservan en su orden de inserción y no cuentan dentro del límite de publicaciones. El método devuelve `list[str]`; la impresión queda a cargo del programa principal.

Actividad 8. Si el usuario tiene 2 publicaciones y sus amigos tienen 5 y 3, ¿cuántos elementos reúne `feed`? Con `limit=4` y 7 comentarios en total entre los cuatro posts elegidos, ¿cuántas líneas devuelve el método contando el encabezado?

1.13 Ejecutar una simulación completa

1.13.1 Archivo social/main.py

Listado 1.12

```
from .services.social_network import SocialNetwork

def demo_social_network() -> None:
    network = SocialNetwork("Social ITLA")
    leo = network.register_user("leo", "Leonardo")
    ana = network.register_user("ana", "Ana")
    maria = network.register_user("maria", "Maria")

    network.add_friends("leo", "ana")
    network.add_friends("leo", "maria")

    p1 = leo.create_post("Mi primer post")
    p2 = ana.create_post("Aprendiendo estructuras de datos")
    p3 = maria.create_post("Hoy practicamos Python")

    p1.add_comment(ana, "Bienvenido, Leo")
    p1.add_comment(maria, "Nos vemos en clase")
    p1.add_comment(leo, "Gracias por comentar")
    p2.add_comment(leo, "Las referencias son importantes")
    p2.add_comment(maria, "Voy a practicar")
    p3.add_comment(leo, "Excelente")

    print("\n".join(network.timeline("leo", limit=10)))

if __name__ == "__main__":
    demo_social_network()
```

Abre una terminal en proyecto_social, la carpeta que contiene social, y ejecuta:

Listado 1.13

```
python -m social.main
```

Si tu sistema utiliza el comando python3, reemplaza python por python3. La opción -m ejecuta el módulo dentro de su paquete y permite resolver las importaciones relativas. No ejecutes directamente social/main.py con este diseño de importaciones.

El bloque `if __name__ == '__main__':` llama a la demostración cuando el módulo se ejecuta como programa. Si otro módulo lo importa, la función queda disponible sin iniciar automáticamente la simulación.

1.13.2 Qué debes observar

El encabezado identifica el timeline de leo y la red Social ITLA. Aparecen tres publicaciones y seis comentarios: diez líneas en total contando el encabezado. Normalmente la publicación de María aparece primero, porque se creó después; si las fechas exactas empatan, se conserva el orden estable descrito anteriormente.

Las fechas dependen del momento de ejecución. La salida tiene esta forma; los puntos suspensivos representan líneas omitidas solo en esta ilustración, no código que debas copiar:

Listado 1.14

```
Timeline de @leo en Social ITLA
AAAA-MM-DD HH:MM UTC | @maria: Hoy practicamos Python
AAAA-MM-DD HH:MM UTC | @leo: Excelente
...
```

'\n'.join(...) une las cadenas usando saltos de línea. Así el programa presenta el resultado de manera legible sin cambiar la responsabilidad del servicio, que únicamente construye las líneas.

1.14 Analizar el costo del proyecto

1.14.1 Operaciones individuales

Operación	Costo temporal habitual	Razón
Consultar un usuario	$O(1)$ promedio	Consulta por clave en users.
Registrar un usuario	$O(1)$ promedio amortizado	Consulta e inserción en un diccionario.
Agregar amistad	$O(1)$ promedio amortizado	Dos consultas y dos asignaciones por clave.
Crear una publicación	$O(1)$ amortizado	Crear el objeto y añadirlo al final de posts.
Agregar un comentario	$O(1)$ amortizado	Crear el objeto y añadirlo al final de comments.

Estas cotas simplifican los costos de cadenas, fechas y asignación de objetos; suponen tamaños acotados para esos datos. En diccionarios, un peor caso desfavorable puede requerir tiempo lineal. En listas, append puede necesitar un redimensionamiento [R2].

1.14.2 El costo del timeline requiere varias variables

Llamaremos F al número de amigos recorridos; P al número total de publicaciones reunidas; K al número de publicaciones seleccionadas, como máximo $\min(\text{limit}, P)$; y C al total de comentarios que pertenecen a esas K publicaciones. L será la longitud total del texto producido.

Etapas	Trabajo principal
Reunir publicaciones	$O(F + P)$: visitar amigos y copiar referencias.
Ordenar publicaciones	$O(P \log P)$ en el peor caso, para P al menos 2.
Aplicar el límite	$O(K)$: construir el corte de la lista.
Construir las líneas	$O(K + C + L)$: recorrer y producir texto.

Por tanto, una cota para el método es **$O(1 + F + P \log(P + 1) + K + C + L)$** , usando $\log(P + 1)$ para incluir $P=0$ y $P=1$. Si suponemos textos de longitud acotada, podemos resumirla como $O(1 + F + P \log(P + 1) + C)$, porque K no supera P . El término F importa incluso cuando muchos amigos no tienen publicaciones.

La lista feed ocupa $O(P)$ referencias y selected ocupa $O(K)$. La ordenación puede usar memoria temporal $O(P)$; lines añade memoria para sus referencias y su texto. La memoria adicional total, incluyendo la salida, es $O(P + K + C + L + 1)$. No duplicamos los objetos Post ni Comment existentes.

Precisión conceptual. Los bucles de publicaciones y comentarios no son automáticamente cuadráticos. Visitan K publicaciones y C comentarios: $O(K + C)$ antes de contar el tamaño del texto. Solo obtendríamos $O(K^2)$ bajo una condición adicional, como tener una cantidad proporcional a K de comentarios por publicación.

1.14.3 Limitar la salida no elimina el trabajo previo

Aunque limit sea 10, esta implementación reúne y ordena todas las P publicaciones candidatas. El límite reduce la representación final, pero no convierte el método completo en $O(10)$ ni en $O(1)$. Para volúmenes grandes podríamos estudiar selección de los K más recientes o mezcla de secuencias ordenadas en capítulos posteriores.

Actividad 9. Hay $F=50$ amigos, $P=1\,000$ publicaciones y $K=10$ publicaciones seleccionadas que suman $C=25$ comentarios. ¿Cuántas publicaciones se ordenan? ¿Cuántas líneas se construyen? Explica por qué limitar a 10 no vuelve constante al algoritmo.

1.15 Comprobar el comportamiento y resolver errores

Antes de ampliar la aplicación, verifica las reglas del modelo. Las siguientes comprobaciones pueden escribirse en una sesión del intérprete iniciada desde `proyecto_social`. Son casos independientes de la demostración principal.

Listado 1.15

```
from social.services.social_network import SocialNetwork

red = SocialNetwork("Prueba")
a = red.register_user("a", "Ana")
b = red.register_user("b", "Beto")
red.add_friends("a", "b")
assert a.friends["b"] is b
assert b.friends["a"] is a
red.add_friends("a", "b")
assert len(a.friends) == 1

p = a.create_post("Hola")
p.add_comment(b, "Saludos")
assert a.posts[0] is p
assert p.comments[0].author is b
assert len(red.timeline("a", 1)) == 3
assert len(red.timeline("a", 0)) == 1
```

Cada `assert` expresa una condición que debe ser verdadera. Si alguna falla, Python lanza `AssertionError`. Para comprobar validaciones, ejecuta por separado `red.register_user('a', 'Otra persona')`, `red.add_friends('a', 'a')`, `red.get_user('nadie')` y `red.timeline('a', -1)`. Cada llamada debe lanzar `ValueError`.

1.15.1 Diagnóstico de problemas frecuentes

Síntoma	Revisión sugerida
Attempted relative import with no known parent package	Ejecuta <code>python -m social.main</code> desde la carpeta que contiene <code>social</code> .
No module named social	Revisa la ubicación de la terminal y el nombre del paquete.
Importación circular o módulo parcialmente inicializado	Revisa las importaciones normales y las protegidas por <code>TYPE_CHECKING</code> .
Un amigo no aparece en ambos usuarios	Revisa las dos asignaciones de <code>add_friend</code> .
feed contiene listas y no publicaciones	Usa <code>extend</code> al reunir posts.
<code>TypeError</code> al ordenar fechas	No mezcles fechas sin zona horaria con fechas que incluyen zona.
Comentarios aparecen en publicaciones ajenas	Revisa que <code>comments</code> se inicialice dentro de <code>__init__</code> .
El timeline no incluye a un amigo de un amigo	Es el alcance previsto: solo se recorren amigos directos.

Actividad 10. Registra un cuarto usuario sin amistad con Leo y crea una publicación suya. Comprueba que no aparece en el timeline de Leo. Luego establece la amistad y vuelve a consultar. Explica el cambio sin hablar de copias de usuarios.

1.16 Práctica integradora

Amplía la red social con una operación **remove_friends(first, second)**. Debe buscar a ambos usuarios, rechazar la autoamistad y retirar la relación de los dos diccionarios. Si la amistad no existe, la operación debe terminar sin modificar el resto de la red ni lanzar un error por esa ausencia.

Antes de escribir código, explica en tus palabras qué datos cambiarán, qué datos seguirán existiendo y qué complejidad esperas. Quitar una amistad no debe eliminar a los usuarios ni borrar sus publicaciones o comentarios.

1.16.1 Secuencia de trabajo

1. Conserva funcionando el proyecto base y ejecuta su demostración.
2. Define la operación que retira la relación en ambos sentidos.
3. Prueba una amistad existente y verifica los dos diccionarios.
4. Repite la eliminación y comprueba que no falla.
5. Consulta el timeline antes y después. Las publicaciones del antiguo amigo dejarán de ser candidatas si no son propias.
6. Comprueba que los comentarios existentes de esa persona permanecen en las publicaciones: eliminar amistad no implica borrar comentarios.

1.16.2 Evidencias de aprendizaje

Entrega el código ejecutable y una explicación breve que justifique el uso de dict y list. Incluye una demostración de identidad con is, un caso de error controlado y el análisis del timeline con F, P, K y C. No basta con indicar «funciona»: muestra qué regla verificaste.

Criterio	Evidencia esperada
Organización de datos	Colecciones apropiadas y responsabilidad clara por clase.
Referencias	Autor y amigos conservan los objetos registrados.
Reglas del modelo	Unicidad, bidireccionalidad y validación de usuarios.
Ejecución	Programa invocable como módulo y salida legible.
Análisis	Distingue promedio, peor caso, tiempo y memoria.
Ampliación	Elimina amistad en ambos sentidos sin borrar los usuarios.

Reto adicional. Añade una búsqueda de publicaciones por una palabra y explica qué colección recorres. Si decides buscar solo en el timeline limitado, documenta que los resultados pueden excluir publicaciones antiguas. El alcance funcional también forma parte del diseño.

1.17 Respuestas y solución orientativa

Intenta resolver las actividades antes de leer estas respuestas. Lo importante es relacionar cada decisión con una operación concreta.

Actividad 1. Un diccionario por código único facilita la consulta por identificador. Para presentar los libros ordenados por título haría falta ordenar una colección o mantener otra organización; buscar por código no resuelve automáticamente ese orden.

Actividad 2. Deshacer: pila, porque se revierte la última acción. Impresión: cola, porque se atiende el orden de llegada. Carpetas: árbol, por la jerarquía. Amistades: grafo, por las conexiones entre personas.

Actividad 3. Ambos tienen tiempo $\Theta(n)$, contando también la copia como otro recorrido lineal. El primero usa $O(1)$ memoria auxiliar; el segundo utiliza $\Theta(n)$ para la copia, bajo el modelo de valores de tamaño acotado.

Actividad 4. a) $2n$ visitas, $\Theta(n)$. b) $3n$ pasos internos, $\Theta(n)$. c) n^2 pares si cada elemento se combina con todos, $\Theta(n^2)$. Si se excluyen repeticiones y se consideran pares no ordenados, serían $n(n-1)/2$ y el orden seguiría siendo cuadrático.

Actividad 5. Ambas referencias observan el name modificado. Si reasignas una variable a otro User, la otra conserva el objeto anterior. Modificar el objeto y cambiar un vínculo son operaciones diferentes.

Actividad 6. `post.py` ejecuta `Comment(...)` para crear comentarios; necesita esa clase al ejecutar. User se usa únicamente como anotación del autor en ese módulo y puede importarse solo para el analizador.

Actividad 7. Ambas expresiones con `is` son `True`. Omitir la segunda asignación dejaría una relación en un solo sentido y rompería la regla de amistad bidireccional.

Actividad 8. Se reúnen 10 publicaciones. El resultado tiene 12 líneas: 1 encabezado, 4 publicaciones y 7 comentarios.

Actividad 9. Se ordenan las 1 000 publicaciones candidatas. Se construyen 36 líneas: $1 + 10 + 25$. El costo de reunión y ordenación sigue dependiendo del volumen completo.

Actividad 10. Antes de la amistad, el cuarto usuario no pertenece a friends de Leo. Después, su objeto se obtiene al recorrer `friends.values()` y sus publicaciones se incorporan a feed. Se comparten referencias; no se duplica al usuario.

1.17.1 Solución orientativa de la práctica

Añade este método a SocialNetwork. `pop(clave, None)` elimina una clave si existe y devuelve `None` si está ausente, evitando un error por amistad inexistente.

Listado 1.16

```
def remove_friends(self, first: str, second: str) -> None:
    user1 = self.get_user(first)
    user2 = self.get_user(second)
    if user1.username == user2.username:
        raise ValueError("No puedes retirarte como amigo.")
    user1.friends.pop(user2.username, None)
    user2.friends.pop(user1.username, None)
```

Con los supuestos de diccionario indicados, el costo esperado de las consultas y eliminaciones es $O(1)$. La memoria auxiliar es $O(1)$. Si un usuario no existe, `get_user` lanza `ValueError` antes de modificar la relación. Las publicaciones y los comentarios permanecen porque no operamos sobre sus listas.

1.18 Síntesis, glosario y fuentes

1.18.1 Lo que debes poder explicar

Al terminar este capítulo deberías poder elegir una estructura según las operaciones del problema; distinguir secuencias, jerarquías y redes; razonar sobre tiempo y memoria; interpretar Big O, Omega y Theta; y construir un pequeño modelo con clases y referencias compartidas.

La red social une esas ideas: los diccionarios aceleran las consultas por identidad, las listas conservan colecciones para recorrerlas y las referencias conectan usuarios, publicaciones y comentarios. El timeline muestra que una salida pequeña puede exigir procesar muchos datos antes de producirse.

1.18.2 Glosario de consulta

Término	Definición en este capítulo
Algoritmo	Secuencia de pasos para resolver una tarea.
Estructura de datos	Organización de información y soporte de sus operaciones.
Referencia	Vínculo que permite acceder a un objeto.
Identidad	Propiedad de ser exactamente el mismo objeto, comprobada con is.
Diccionario	Asociación de claves con valores.
Grafo	Vértices conectados por aristas; aquí, usuarios y amistades.
Invariante	Regla que debe mantenerse tras las operaciones válidas.
Cota asintótica	Límite de crecimiento para entradas suficientemente grandes.
Costo amortizado	Costo distribuido sobre una secuencia de operaciones.
Timeline	Publicaciones propias y de amigos directos, ordenadas por fecha.

1.18.3 Materiales base del capítulo

[S1] Presentación de fundamentos de estructura de datos. Material de clase de Ing. Leonardo Tavarez, 15 diapositivas. Base de las secciones sobre definición, clasificación, operaciones, patrones y complejidad.

[S2] Transcripción suministrada: Pasted text(20260910-163528).txt. Clase práctica sobre clases y referencias en Python mediante una red social. Base del proyecto con Comment, Post, User y servicio de red social.

Este manual reorganiza ambos materiales como una lección autónoma. Los ejemplos, actividades, comprobaciones y la ampliación final son desarrollos didácticos. Se precisaron las diferencias entre cotas y casos de análisis, las capacidades de `dataclass` y el costo del `timeline`. La transcripción termina durante la explicación del comando de ejecución; aquí se completa con una estructura de paquetes coherente.

1.18.4 Referencias técnicas de apoyo

[R1] Cornell University. Lecture 12: Asymptotic complexity. Definiciones de O, Omega y Theta.

<https://www.cs.cornell.edu/courses/cs312/2003sp/lectures/lec12.html>

[R2] Python Wiki. TimeComplexity. Costos de operaciones en listas y diccionarios de CPython.

<https://wiki.python.org/moin/TimeComplexity>

[R3] Python Software Foundation. The Python Tutorial: Classes. Objetos, referencias y métodos.

<https://docs.python.org/3/tutorial/classes.html>

[R4] Python Software Foundation. typing: Support for type hints. Anotaciones y `TYPE_CHECKING`.

<https://docs.python.org/3/library/typing.html>

[R5] Python Software Foundation. `dataclasses`, documentación de Python 3.10. Opciones y generación de métodos.

<https://docs.python.org/3.10/library/dataclasses.html>

[R6] Python Software Foundation. Sorting Techniques. Claves de ordenación, `reverse` y estabilidad.

<https://docs.python.org/3/howto/sorting.html>

Referencias consultadas el 10 de septiembre de 2026. El código usa sintaxis compatible con Python 3.10 o posterior y únicamente la biblioteca estándar.